environments becomes that much more complicated with containerization. When working in cloud computing, with its premade bundles of operating systems and libraries, you often need to inject configuration information from the outside. Appendix B touches on a few options for these situations, but there are also third-party libraries out there that can make managing multiple configurations easier:

- *Database URLs*—Different system environments need to connect to different databases and, sometimes, even different types of databases. The dj-database-url (https://pypi.org/project/dj-database-url/) library allows you to specify database configurations in a single URL, which can be injected into your project through an environment variable.
- *Django configurations*—The need to have different settings for each environment is one that has been solved several ways by third-party libraries. The django-configurations (https://pypi.org/project/django-configurations/) tool kit allows you to create classes inside of settings.py that group configurations together. You then use an environment variable to specify which class gets instantiated.
- *Django environ*—Another approach to the settings problem is django-environ (https://pypi.org/project/django-environ/). As the name implies, it provides tools for getting configuration values out of environment variables. This is a common approach when dealing with container systems, like Kubernetes.

14.3 Dealing with data

A core element of dynamic websites is the data they store, manipulate, and then present to the users. There are many kinds of data you might use, and there are libraries out there to better define, store, and interact with it in your projects:

- *Money*—Representing money in software is more problematic than you might first think. A naive approach is to use floating-point numbers for currency, but that creates inaccuracies. Floating-point numbers do not contain an exact decimal representation. Open a REPL and try `0.1 + 0.2` to see what I mean (`0.30000000000000004`). The Python standard library includes the `Decimal` class, which does have exact representation, and the third-party library django-money (https://pypi.org/project/django-money/) can be a big help. It comes with a `Model` field for storing monetary values and even includes a currency attribute to handle exchange.
- *Import and export*—If you have data in your system, you will likely find yourself needing to get data into and out of it on a regular basis. The django-import-export (https://pypi.org/project/django-import-export/) library provides a series of Django Admin tools for importing and exporting `Model` data. It supports CSV, JSON, Excel, and many other formats, and it even has a preview mode where you can see the proposed changes before enacting them.
- *Tagging*—One way of categorizing data is to tag it. This approach is quite common in publication systems, where each article has one or more tags that indicates what it is about, allowing users to easily find other articles on the same